Credit Risk Workbench: how to build it

Ideation, architecture, design and a review of the design

risk-workbench

25 October 2026. Source: `credit-risk-workbench-design-brief.md` v1.0

The answer

1. **The data layer becomes a contract, not a database.** The brief's Section 9 tables become canonical tables. Each table has fixed columns, units, a sign rule and two clocks: `effective_at` (when it was true) and `available_at` (when it was known).
2. **One narrow port reads them.** A `Source` has one method: `read(kind, window)`. This machine plugs in a synthetic source and a parquet-tables source. The work machine plugs in its own adapter by setting `CRW_SOURCE=module:factory`. No other code changes.
3. **Every run freezes what it read.** The run copies its inputs into an archive. The archive is itself a `Source`, so any old report replays exactly, even after the work database revises.
4. **The calculations never touch the data layer.** They are pure functions on canonical tables. Orchestration reads, calls them, and writes results. The screens read results only.
5. **This machine proves the method on synthetic data.** It has no book and no official P&L. A synthetic market with known factor moves is the test oracle. The brief's Section 10 checks become the acceptance tests.

The review rates the design sound, with three things to fix before code (Section 4). The most important: the read-recorder must not be shared across parallel workers.
Agreed in review (Section 5, overrides earlier sections): semi-live runs (start of day, refresh, end of day); CP+ intraday prices with per-bond staleness; FINCAD analytics behind a new `Pricer` port. Positions and trades are still open.

Contents

1 Ideation **2**
- 1.1 The problem, stated precisely . . . 2
- 1.2 Three ways to build it . . . 2
- 1.3 Recommendation: A's port with B's contract . . . 3
- 1.4 Assumption audit . . . 4
- 1.5 Inversion: how would this project fail? . . . 4
- 1.6 Second-order effects . . . 5
- 1.7 The likely change in six months . . . 5
- 1.8 Parallel and sequential work . . . 5

2 Architecture **6**
- 2.1 Domain concepts . . . 6
- 2.2 How the concepts relate . . . 6
- 2.3 Rate of change . . . 6
- 2.4 Modules . . . 7
- 2.5 Abstraction decisions . . . 7
- 2.6 Dependency direction . . . 7

3 Design **7**
3.1 Objective and core abstraction . . . 7
3.2 Components . . . 8
3.3 Dependency graph . . . 8
3.4 Data flow for one day . . . 8
3.5 Mathematical contract . . . 9
3.6 Key interfaces . . . 9
3.7 Configuration . . . 10
3.8 Errors . . . 10
3.9 Testing . . . 11
3.10 File structure . . . 11
3.11 Build plan . . . 11
3.12 Risks and what we deliberately skip . . . 12

4 Architecture review of this design **12**

5 Agreed in review, 25 October 2026 **13**
5.1 The production machine has . . . 13
5.2 Changes to the design . . . 13
5.3 Still open . . . 14
5.4 First steps on the production machine . . . 14

1 Ideation

1.1 The problem, stated precisely

Goal. One code base computes the brief's measures, attribution and scenarios. It runs here on synthetic and partial public data. It runs on the work machine on the desk's real book, marks and analytics. The move needs one new adapter and no edits to the calculations.

What the goal is not. It is not "support any database". It is "the calculations depend on a stated data contract, and each machine meets that contract its own way".

What this machine has and lacks.

- **Has:** TRACE prints (May 2023 to Sep 2025, with duplicate rows), Databento ETF quotes with true receive times, WRDS access, the Treasury curve.
- **Lacks:** positions, trades, official P&L, vendor OAS and spread duration, ETF holdings history. These are the inputs that Layers 0 and 1 need first.

So the local work is method work. Real local data can feed the market monitor later. It cannot validate P&L attribution.

1.2 Three ways to build it

A. Ports per kind of data. Narrow Python protocols, one per data family. Each machine writes adapters. This is the pattern already in use in `FundamentalQuantmental/hy/data`.
Implication: the contract lives in Python signatures. *Risk:* adapters drift on units and timing, because a signature cannot say "decimal, not bp". *Change cost:* a new kind means a new method on every adapter.

B. Canonical store first. Each machine runs ingest jobs that write a local store in the canonical schema. The calculations read only that store.
Implication: the contract lives in table schemas. *Risk:* a second copy of the work data must be kept fresh. A late correction at work needs a re-ingest. *Change cost:* a new kind means a new table and a new ingest job.

C. Wrap the work risk system. Use the desk's vendor analytics for exposures and P&L. Compute only the missing pieces.
Implication: fastest at work. *Risk:* nothing runs here, and the method cannot be tested. It also breaks principle 1 of the brief: market measures must be independent of the book and of one vendor's model.

	A. Ports	B. Store first	C. Wrap vendor
Complexity	Low	Medium (ingest jobs)	Low at work, none here
Portability	Good: one adapter	Good: one ingest job	Poor
Unit and timing safety	Weak unless checked	Strong: one schema	Vendor-defined
Reproducibility (Layer 6)	Needs archiving	Built in	Vendor-dependent
Testability	Strong (fakes)	Strong (fixture tables)	Weak
Time to first result	Short	Medium	Short at work only
Fit with your other repo	Same pattern	Partly (`tables` source)	None

1.3 Recommendation: A's port with B's contract

Take the narrow port from A. Take the canonical schema from B. Add one piece neither has: **snapshot on read**.

1. The canonical schema (columns, units, signs, two clocks) is the contract. It lives in the domain package.
2. One port, `Source.read(kind, window)`, returns canonical rows. It is one method, so a fake takes five lines.
3. A `View` wraps any source. It checks the schema, removes rows not yet known at the as-of time, keeps the latest revision, and raises `DataUnavailable` instead of returning zeros.
4. The view records every frame it served. The run writes them to its archive. An `ArchiveSource` replays a run.
5. A contract checker reads every kind through the view and reports wrong units, wrong signs, look-ahead and gaps. It is the first thing to run on the work machine.

This keeps A's low cost. It gets B's safety without a second data copy at work. It meets the brief's Layer 6 gate ("reproduce a historical report from archived inputs") by design.

1.4 Assumption audit

Assumption	Weight	Defence in the design
The work data can be mapped into the Section 9 tables	**Load-bearing**	Contract checker; partial sources allowed
Units and signs can be converted at the adapter	**Load-bearing**	Schema states units; checker flags bp-as-decimal, price per 1 vs 100, short sign
Work sources give or allow an `available_at`	**Load-bearing**	Adapter may stamp ingest time; view enforces it
Synthetic data can validate the method	**Load-bearing**	Synthetic market has known factor moves; recovery is the test
Definitions can be written as rules, not ID lists	**Load-bearing**	Baskets select by rating, sector, tenor; never by CUSIP
Sector taxonomy differs by machine	Real	A `taxonomy` kind maps source codes to canonical sectors
The work machine allows pandas, numpy, pyarrow, pyyaml	Real	No other runtime dependency in the core
End-of-day first; intraday later	Cosmetic	Trades keep timestamps; subperiods are a later change
One currency first	Cosmetic	Currency column exists; checker rejects a second one

1.5 Inversion: how would this project fail?

1. **The work adapter returns spreads in bp, and nobody notices.** Every DTS number is 10,000 times too large. *Defence:* the checker flags an OAS above 1.0 as "probably bp". The view refuses frames that fail required checks.
2. **A historical run uses today's ratings.** Quality P&L is then wrong in a way that looks plausible. *Defence:* two clocks on every row. The view filters by `available_at`. A test leaks one future row and must fail.
3. **Missing marks become zero moves.** The residual then looks small and false. *Defence:*

every result row carries a status (observed, calculated, estimated, stale, unavailable). A missing input propagates as `unavailable`.

4. **The calculations grow imports of the work adapter.** The move to the work machine then forks the code. *Defence:* calculation packages may not import `crw.data`. An import-lint test enforces it.
5. **The method is "validated" on the same synthetic data it was tuned on.** *Defence:* the synthetic generator takes a seed. Tests use a held-out seed and the brief's named scenarios, not a fit score.
6. **Old reports change meaning after a definition edit.** *Defence:* definitions are versioned files. The run manifest stores their content hash.

1.6 Second-order effects

- **One generic `read(kind)` port.** Adding a kind adds a schema and an enum value. Adapters that do not offer it simply do not list it. And then: a kind-specific query, such as "holdings of one ETF only", must go through the window and a filter. That is acceptable at HY universe size (about 2,000 bonds).
- **Pure calculations on DataFrames.** Calculations are easy to test and to parallelise. And then: column names become a shared vocabulary. They must live as constants in one place, or they drift.
- **Snapshot on read.** Replay is exact. And then: archives grow. One day of a 2,000-bond universe is a few MB of parquet. A year is about 1 GB. Acceptable.

1.7 The likely change in six months

Most likely: **a second work data system** (for example positions from the book system, analytics from a vendor). The design handles it with a `CompositeSource` that routes each kind to one source. No calculation changes.

Second most likely: **a new factor block** (liquidity, or a second curve point). The design handles it with one new `FactorBlock` class and one definition file.

1.8 Parallel and sequential work

- **Parallel:** each day's exposures, factor estimation and attribution. Opening positions come from data, not from the previous day's result. Basket weights per rebalance date. Scenarios per shock.
- **Sequential:** position roll-forward (each day needs the last). Cumulative sums. Covariance estimation over a factor history.
- **Boilerplate:** each adapter, each canonical schema, each factor block. These are scaffold candidates.

Decision summary

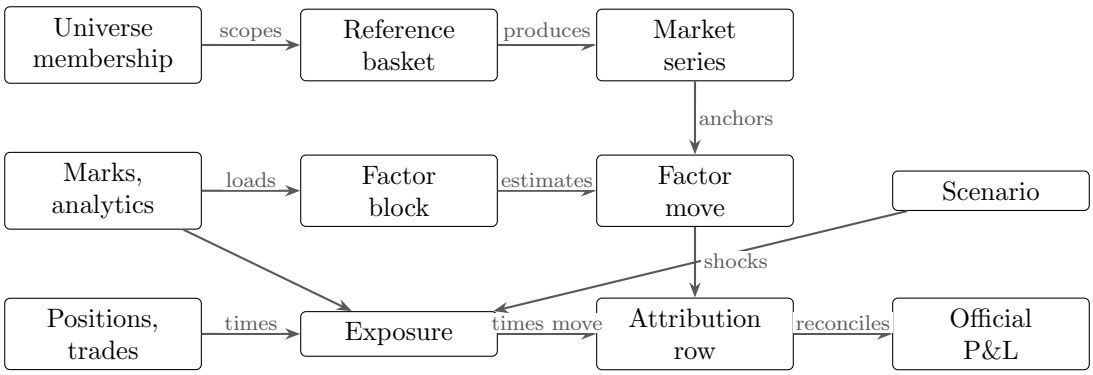
- **Chosen approach:** a canonical schema as the contract, read through one narrow `Source` port and a point-in-time `View`, with every run archiving what it read.
- **Trade-off accepted:** DataFrames cross boundaries, checked by schema at the view, instead of typed records per row.
- **First thing to build:** the schema, the port, the synthetic source, the contract checker, and the Section 10.1 quality-pair check on CS01 and dollar DTS.
- **Extensibility vector:** new work systems become new adapters behind a composite source.

2 Architecture

2.1 Domain concepts

Concept	Type	What it is	Changes when
Security, Issuer	Entity	Reference data with history	Rarely
Position, Trade, Event	Entity	The book and its changes	Daily
Mark, Analytics, Curve	Observation	Prices and sensitivities	Daily
ETF market, ETF holdings	Observation	Price, NAV, distributions, look-through	Daily
Universe membership	Collection	External reference universe	Monthly rebalance
Reference basket	Definition	Rule-based legs, weights, controls	On definition change
Market series	Computed	Basket levels and moves	Daily
Factor block	Definition	Loadings for one factor family	On model change
Factor move	Computed	Daily factor return	Daily
Exposure	Computed	Signed CS01, dollar DTS, factor exposure	Daily
Attribution row	Computed	Dollar contribution per security and component	Daily
Scenario	Definition	Shock, unit, anchor, what stays fixed	On definition change
Run, Manifest	Event	One execution with inputs and versions	Every run
Reading status	Value	Observed / calculated / estimated / stale / unavailable	Never (fixed vocabulary)

2.2 How the concepts relate



2.3 Rate of change

- **Stable:** the canonical schema, units and signs, reading statuses, the `Source` port.
- **Moderate:** basket, factor and scenario definitions; the attribution model version; calculation code.
- **Volatile:** the data adapters (they differ by machine), the data itself, the screens.

The boundaries follow from this. The volatile adapters sit at the edge. The stable schema sits at the centre. Definitions are files, so they change without code changes.

2.4 Modules

Module	Responsibility	Knows about	Does not know about	Changes when
`crw.domain`	The shared vocabulary	Schemas, units, signs, statuses, run context, definition types	Any data source, any calculation	The brief's contract changes
`crw.data`	Reading data safely	The port, the view, adapters, the checker	What calculations do with data	A machine or vendor changes
`crw.book`	Positions and accounting	Roll-forward, reconciliation, ETF look-through	Factors, baskets, storage	Accounting rules change
`crw.exposure`	Sensitivities	CS01, dollar DTS, key rates, aggregation	Where marks come from	Sensitivity conventions change
`crw.baskets`	Market measures	Eligibility, weights, caps, pair normalisation, gaps and ratios	The book	A basket definition changes
`crw.factors`	Factor moves	Factor blocks, loadings, the estimator	Positions, P&L	The attribution model changes
`crw.attribution`	Explaining P&L	Carry, rates, factor, ETF layers, residual, bridge	Storage, screens	Attribution rules change
`crw.results`	Storing outputs	Result schemas, the result store	How results were computed	An output table is added
`crw.runs`	Running a day	Order of steps, manifest, archive, wiring	Calculation internals	The daily process changes
`crw.app`	Screens (later)	Result tables	Calculations and sources	The UI changes

Later layers add `crw.diagnostics` (Layer 4) and `crw.risk` (Layer 5). The review below says not to create them yet.

2.5 Abstraction decisions

- **Protocol** for `Source`: there are three implementations from day one (synthetic, tables, archive), and the work adapter is the fourth.
- **Protocol** for `FactorBlock`: common, quality, sector and curve blocks vary in how they build loadings.
- **Protocol** for `ResultStore`: parquet store and in-memory store for tests.
- **Class** for `View`: it holds the source, the as-of time and the read recorder.
- **Functions** for every calculation: CS01, DTS, basket returns, attribution. They are stateless.
- **Dataclasses** (frozen) for `RunContext`, `Window`, definitions and the manifest.
- **Config** as YAML files loaded into frozen dataclasses (see Section 3.7).

2.6 Dependency direction

All arrows point to `crw.domain`. Calculation packages import only `crw.domain` and each other in one direction: `book` → `exposure` → `factors` → `attribution`, and `baskets` → `factors`. Only `crw.runs` imports `crw.data` and `crw.results`. Nothing imports `crw.runs` except the CLI. The graph is a DAG.

3 Design

3.1 Objective and core abstraction

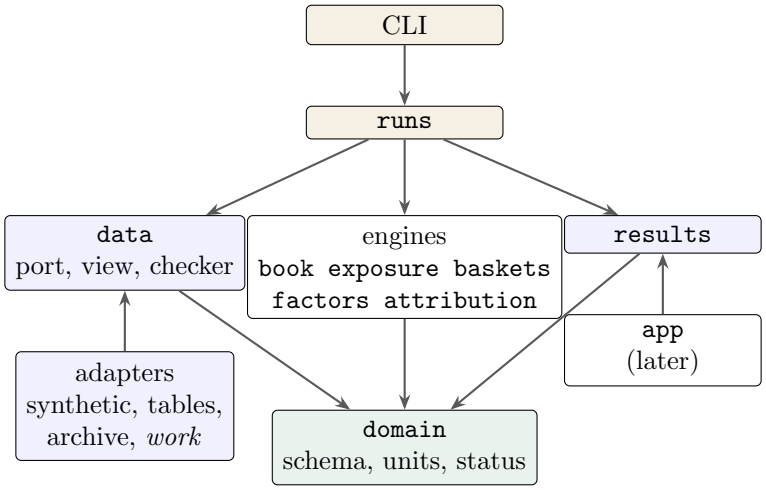
Objective. Turn one day's canonical inputs into exposures, market measures, an additive P&L explanation and scenario losses, each with a status and a version, reproducible from its archive.

Core abstraction: the canonical table read as of a time. Everything else orbits it. A calculation never asks "where is the data". It receives "these rows, known by this time, in these units". If the requirements change, new kinds join the schema. The abstraction holds.

3.2 Components

Component	One responsibility	Uses	Used by
Domain core	Defines the canonical vocabulary	nothing	all
Data access	Serves validated point-in-time tables	domain	runs
Measurement engines	Compute exposures, measures, attribution	domain	runs
Results store	Persists outputs with status and version	domain	runs, app
Run orchestration	Wires one day's steps and archives them	data, engines, results	CLI
Presentation (later)	Shows results	results	user

3.3 Dependency graph



Engines never import `data` or `results`. Adapters depend on the port; the port does not know any adapter. The CLI picks the adapter through `CRW_SOURCE`.

3.4 Data flow for one day

1. **Open.** `runs` builds a `RunContext`: book, currency, interval $[t_0, t_1]$, the opening position time, the current position time, the attribution version, the as-of time.
2. **Read.** The view reads each kind for the window. *Format:* pandas DataFrame with the canonical columns. *Failure:* schema violation raises; a missing kind raises `DataUnavailable`; a row known after the as-of time is dropped, and a direct request for it raises `LookAheadError`.
3. **Book.** Roll positions forward from trades and events. Reconcile to official P&L. Build ETF look-through holdings. *Output:* positions with a `position_basis` column (actual or synthetic).
4. **Exposure.** CS01, $K_i = V_i D_i^s s_i$, key rates, per security, then cash long, cash short, ETF and net.
5. **Measures.** Basket weights at t_0, basket levels and moves at t_1, gaps and log ratios.
6. **Factors.** Loadings at t_0 from each block. Common move m_t from the anchor basket. Remaining factor moves from the estimator.
7. **Attribution.** Per security first: carry, rates, each factor, ETF layers, residual. Then aggregate. Then the bridge: actual P&L = components + reconciliation difference.
8. **Write.** Each result table goes to the store with `run_id`, `model_version` and a status per row. The manifest lists inputs, definition hashes and exceptions. The recorded input frames go to the archive.

Failure policy. A step that fails marks its outputs `unavailable` and the run `incomplete`. It never writes zeros. A rerun with late data writes a new `run_id` that names the run it revises.

3.5 Mathematical contract

Spreads are decimal in all calculations. Display converts to bp.

$$CS01_i = V_i D_i^s 10^{-4}, \qquad K_i = V_i D_i^s s_{i,0}, \qquad z_i = \frac{\Delta s_i}{s_{i,0}}, \qquad \Delta P_i^{credit} \approx -K_i z_i.$$

Factor model (initial). With $\beta = 1$ and the common move m_t fixed by the anchor basket:

$$z_{i,t} - m_t = q_{i,t-1}^\top Q_t + a_{i,t-1}^\top S_t + c_{i,t-1}^\top C_t + u_{i,t}.$$

Weighted least squares across the reference universe each day. Quality uses B as the reference group, so $Q = (Q_{BB/B}, Q_{CCC/B})$ are two contrasts. Sector effects sum to zero under the universe's DTS weights, so they cannot carry the common move. Factor exposure and P&L are

$$E_{k,t-1} = \sum_i K_{i,t-1} x_{ik,t-1}, \qquad P\&L_{k,t} \approx -E_{k,t-1} f_{k,t}.$$

Where it breaks.

- **Empty cell.** A sector with no names that day has no column. Its factor gets status `unavailable`, not zero.
- **Singular design.** The sum-to-zero constraint plus the reference group removes the redundancy. The estimator checks the rank and refuses if it is short.
- **Distressed bonds.** Above a definition-set spread or below a price threshold, a bond leaves the proportional model. Its P&L goes to an explicit "price-mode exception" line.
- **Overfitting.** None is estimated across days in the first model. The reserved validation dates from the brief's P0 task stay out of every tuning step.

3.6 Key interfaces

```python
from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime
from enum import Enum
from typing import Protocol, runtime_checkable

import pandas as pd


class Kind(str, Enum):
    """The canonical tables (brief Section 9). Schemas live in crw.domain.schema."""
    SECURITIES = "securities"
    UNIVERSE = "universe"
    TAXONOMY = "taxonomy"
    POSITIONS = "positions"
    TRADES = "trades"
    OFFICIAL_PNL = "official_pnl"
    MARKS = "marks"
    ANALYTICS = "analytics"
    CURVES = "curves"
    ETF_MARKET = "etf_market"
    ETF_HOLDINGS = "etf_holdings"


@dataclass(frozen=True)
```

```python
class Window:
    """Rows whose effective_at lies in [start, end]."""
    start: datetime
    end: datetime


@runtime_checkable
class Source(Protocol):
    """One machine's data. Returns every revision of canonical rows, each with
    effective_at and available_at. Point-in-time filtering is the View's job."""

    def kinds(self) -> frozenset[Kind]: ...
    def read(self, kind: Kind, window: Window) -> pd.DataFrame: ...


@runtime_checkable
class FactorBlock(Protocol):
    """One family of attribution factors (common, quality, sector, curve)."""

    block_id: str

    def loadings(self, cross_section: pd.DataFrame) -> pd.DataFrame:
        """Index security_id, one column per factor_id, from data known at t0."""
        ...


@runtime_checkable
class ResultStore(Protocol):
    """Where a run's output tables go. Rows carry run_id, model_version, status."""

    def write(self, table: str, frame: pd.DataFrame, run_id: str) -> None: ...
    def read(self, table: str, run_id: str) -> pd.DataFrame: ...
```

The view is a concrete class: `View(source, as_of, recorder).get(kind, window)`. It validates the schema, drops rows with `available_at` after `as_of`, keeps the latest revision per key, and records the frame it returns.

3.7 Configuration

Approach: Dataclasses. YAML files load into frozen dataclasses with explicit validation. This overrides the design skill's Pydantic default. Reasons: your HY Analyst repo uses the same approach, and the work machine may restrict packages. Each extra runtime dependency is one more approval at work.

- `definitions/baskets/*.yaml`, `definitions/factors/*.yaml`, `definitions/scenarios/*.yaml`: each has `id` and `version`. The manifest stores a content hash.
- `config/run.yaml`: book, currency, universe id, data root, as-of rule.
- Environment: `CRW_SOURCE` (adapter), `CRW_DATA_ROOT` (tables and archive), `CRW_RESULTS_ROOT`.

3.8 Errors

- **Data layer:** raise on schema violation, `DataUnavailable`, `LookAheadError`. Adapters may retry network reads. The view never fills gaps.
- **Engines:** a missing input for one security gives that security's rows status `unavailable`. They do not stop the day.
- **Runs:** catch per step, mark the run `incomplete`, write the exception to the manifest, exit non-zero.

3.9 Testing

- **Unit:** every formula in Section 8 of the brief, including the 10.1 example ($0 and −$10,000) and the ETF identity.
- **Acceptance:** one test per row of the brief's Section 10 table, run on the synthetic source.
- **Recovery:** the synthetic market draws known factor moves; the estimator must recover them within a tolerance on a held-out seed.
- **Contract:** the checker passes on the synthetic and tables sources; a deliberately broken source (bp spreads, future rows, positive short) must fail each check.
- **Boundary:** an import test fails if any engine imports `crw.data`, `crw.results` or `crw.runs`.
- **Replay:** a run, then a replay from its archive, gives identical results.

3.10 File structure

```
risk-workbench/
|-- pyproject.toml
|-- config/run.yaml
|-- definitions/            baskets/, factors/, scenarios/  (versioned YAML)
|-- docs/design/            this document; data_contract.md; work_machine.md
|-- src/crw/
|   |-- domain/             schema.py  units.py  status.py  context.py
|   |                       definitions.py  errors.py
|   |-- data/               source.py  view.py  factory.py  contract.py
|   |   `-- adapters/       synthetic.py  tables.py  archive.py  composite.py
|   |-- book/               positions.py  reconcile.py  lookthrough.py
|   |-- exposure/           sensitivities.py  aggregate.py
|   |-- baskets/            weights.py  measures.py
|   |-- factors/            blocks.py  estimate.py
|   |-- attribution/        credit.py  carry_rates.py  etf.py  bridge.py
|   |-- results/            store.py  tables.py
|   |-- runs/               daily.py  manifest.py
|   `-- cli.py
`-- tests/                  unit/  acceptance/  contract/  test_boundaries.py
```

3.11 Build plan

Each step ends with something usable, as the brief requires. Times are agent working time, assuming no surprises.

Step	Build	Usable output	Time
M0	Domain schema, port, view, factory, checker, synthetic and tables sources, result store, manifest	`crw check-source` passes on synthetic data	1 day
M1 (Layer 0)	Roll-forward, reconciliation, CS01, dollar DTS, key rates, cash/ETF/net	Position and issuer report with coverage	1–2 days
M2 (Layer 1)	Market diary, carry and rates, common credit, ETF layers, residual	"Did the hedge offset the common move?"	2 days
M3 (Layer 2)	Basket engine, B/BB, CCC/B, largest sector pairs	Market monitor with drill-down	2–3 days
M4 (Layer 3)	Factor blocks, estimator, bridge, versioning	Additive P&L bridge	3 days
Later	Layers 4–6, screens, work adapter		

M0 to M4 are file-disjoint enough for `orchestrate` after M0 lands. M0 itself should be one agent, because every later step depends on the schema.

3.12 Risks and what we deliberately skip

- **Risk:** the canonical schema is wrong in a way only the work data reveals. *Mitigation:* write `docs/design/data_contract.md` early, and take it to work before M2.
- **Risk:** the local fallback pricer (spread duration from yield and maturity) is crude. It is only for synthetic and TRACE data. At work, vendor analytics replace it.
- **Skipped:** screens, intraday, multi-currency, covariance, hedge comparison. The brief puts them later.
- **Skipped:** a local real-data adapter for TRACE. Its duplicate rows make it unfit for attribution tests.

4 Architecture review of this design

No code exists yet, so this review scores the proposed structure, not files. It uses the seven dimensions of the review-architecture skill.

Dimension	Score	Key finding
Boundary quality	Strong	Modules follow rate of change; `attribution` needs watching
Dependency direction	Strong	All arrows point to `domain`; enforced by a test
Abstraction fitness	Adequate	Generic `read(kind)` trades type safety for portability
DRY and knowledge	Adequate	Column names and units are shared strings: one home needed
Extensibility	Strong	New source: 1 new file. New factor: 1 file plus 1 YAML
Testability	Strong	Engines are pure; runs needs 3 fakes
Parallelisation	Adequate	Days are independent, but the read recorder is shared state

Overall: sound. Fix three things before code.

Fix before code

AR-PAR-001 (Adequate). `View` holds a recorder that collects every frame read. If days run in parallel workers, one shared recorder is mutable shared state. *Direction:* one view and one recorder per day per worker. The run merges archives after the workers finish.

AR-DRY-001 (Adequate). Engines, the checker and the result store all use column names and units. As strings, they drift. *Direction:* `crw.domain.schema` owns every column name, dtype and unit. Nothing else writes a literal column name.

AR-ABS-001 (Adequate). `read(kind, window)` returns an untyped frame. A wrong frame fails late. *Direction:* the view validates against the kind's schema on every read, so the failure happens at the boundary. Accept the trade-off; do not add per-kind methods.

Watch during build

AR-BND-001 (Adequate). `attribution` will gather carry, rates, factors, ETF layers, residuals and the bridge. *Direction:* keep the bridge as the only module that sees all components; split further if a file passes 300 lines.

AR-BND-002 (Adequate). Basket rules name sectors. Sector codes differ by machine. *Direction:* definitions use canonical sector codes only; the `taxonomy` kind maps source codes to them.

AR-EXT-001 (Weak, inherent). A new canonical kind touches the schema, the checker, the tables adapter and the synthetic adapter: four files. *Direction:* accept it, because kinds change rarely. Generate the schema-driven parts in the tables adapter from the schema, so it needs no edit.

AR-ABS-002 (Adequate). Do not create `diagnostics` or `risk` packages before Layers 4 and 5. Empty packages invite misplaced code.

What to keep

- Snapshot on read: it turns reproducibility from a process into a property.
- Status on every result row: it makes "missing is not zero" structural, not a UI convention.
- The same port shape as `hy.data`: one mental model across both repos.

5 Agreed in review, 25 October 2026

These points came out of the discussion of this document. They **override** earlier sections where the two differ.

5.1 The production machine has

- **Intraday bond prices from MarketAxess CP+**, at regular intervals, with a timestamp per bond.
- **FINCAD** for analytics (OAS, spread duration, key rates), callable directly from Python.

5.2 Changes to the design

1. **The data-layer cut stands.** One canonical schema, one `Source.read(kind, window)` port, a point-in-time view, snapshot on read.
2. **Three run modes, not one daily run.**
   - *Start of day:* opening positions, sensitivities, factor loadings and basket weights, computed once.
   - *Refresh:* every few minutes, latest prices → factor moves → P&L. Only this part reruns.
   - *End of day:* the last refresh plus the official P&L reconciliation.

   `RunContext.as_of` is a datetime, not a date. Semi-live is built after M2 (about 2–3 days). The start-of-day/refresh split is built from M0 (about 1 extra day).
3. **Prices are a stream.** The view takes, per bond, the latest price at or before the refresh time. A price may be used only once it has *arrived* (`available_at`). Its age is measured from *its own* timestamp (`observed_at`).
4. **Stale is per bond, and not only by age.** CP+ publishes on a schedule, so a fresh timestamp can carry a model-carried price. A bond is flagged stale when (a) its price age exceeds a limit set in a settings file (start: one limit, for example 30 minutes), or (b) its price has not changed for a set number of intervals, or (c) a CP+ quality field, if the feed carries one, says so. A stale bond's P&L goes on a stale line, never to zero.
5. **Intraday moves use fresh bonds only**, and show their coverage (for example "1,240 of 1,900 repriced"). Liquid bonds reprice more often, so coverage must sit next to every intraday move.
6. **Price moves become spread moves by a labelled first-order rule** until FINCAD repricing is fast enough: price change since the open, minus the Treasury move times the opening rate sensitivity, divided by the opening spread duration. CP+ benchmark spreads are not OAS and are not mixed with FINCAD OAS.
7. **ETF prices are taken at the refresh time**, and a gap between bond and ETF timestamps is flagged.
8. **Intraday positions** are opening positions plus trades so far. P&L is split at each trade time.

9. **Price source: CP+ for both refresh and close, set in settings.** Each price row carries a `source` column. Settings name the refresh source and the close source. A "price source change" P&L line switches on only when they differ. A gap to the desk's official P&L shows on the reconciliation line.

10. **A pricer port joins the design.**

```python
@runtime_checkable
class Pricer(Protocol):
    """Analytics for bonds at given prices. Spreads are decimal; durations positive."""

    def analytics(self, terms: pd.DataFrame, prices: pd.DataFrame,
                  curve: pd.DataFrame) -> pd.DataFrame: ...
```

- Chosen by `CRW_PRICER=module:factory`, the same pattern as `CRW_SOURCE`.
- Here: a simple local pricer for plain fixed-coupon bonds; callables get status `estimated`.
- At work: a FINCAD adapter.
- Start of day: FINCAD output is stored and archived as the `analytics` table.
- Scenarios and callables use full repricing through the port. Intraday full repricing replaces the first-order rule once its speed is measured at work.
- A **pricer check** prices a set of plain bonds with known answers. It checks that OAS is decimal, spread duration is positive, and key rates sum to about the duration.
- Pricing runs bond by bond. It is sequential by default, because the FINCAD licence may not allow parallel calls. A setting turns parallel on.

5.3 Still open

1. **Positions and trades:** which system or systems at work. If several, a `CompositeSource` routes each kind.
2. **Package name:** `crw` proposed, not confirmed.
3. **Dataclasses or Pydantic:** dataclasses proposed, not confirmed.
4. **Section 13 defaults for M0–M2:** proposed USD only, ICE-style HY universe rules, B as the quality reference group. Not confirmed.
5. **CP+ fields:** whether the feed carries a quality or confidence field (used by the stale rule).
6. **FINCAD speed** for about 2,000 bonds, which decides intraday full repricing.

5.4 First steps on the production machine

1. Answer the open items above, starting with positions and trades.
2. Build M0 there, including the start-of-day/refresh split and the pricer port.
3. Write the CP+ source adapter and the FINCAD pricer adapter.
4. Run the data check and the pricer check. Fix every failed required row before M1.